PROJECT IMPLEMENTATION GUIDE

Infrastructure Monitoring and Visualization Using Prometheus & Grafana

Document created by Richard L

Architecture and component roles

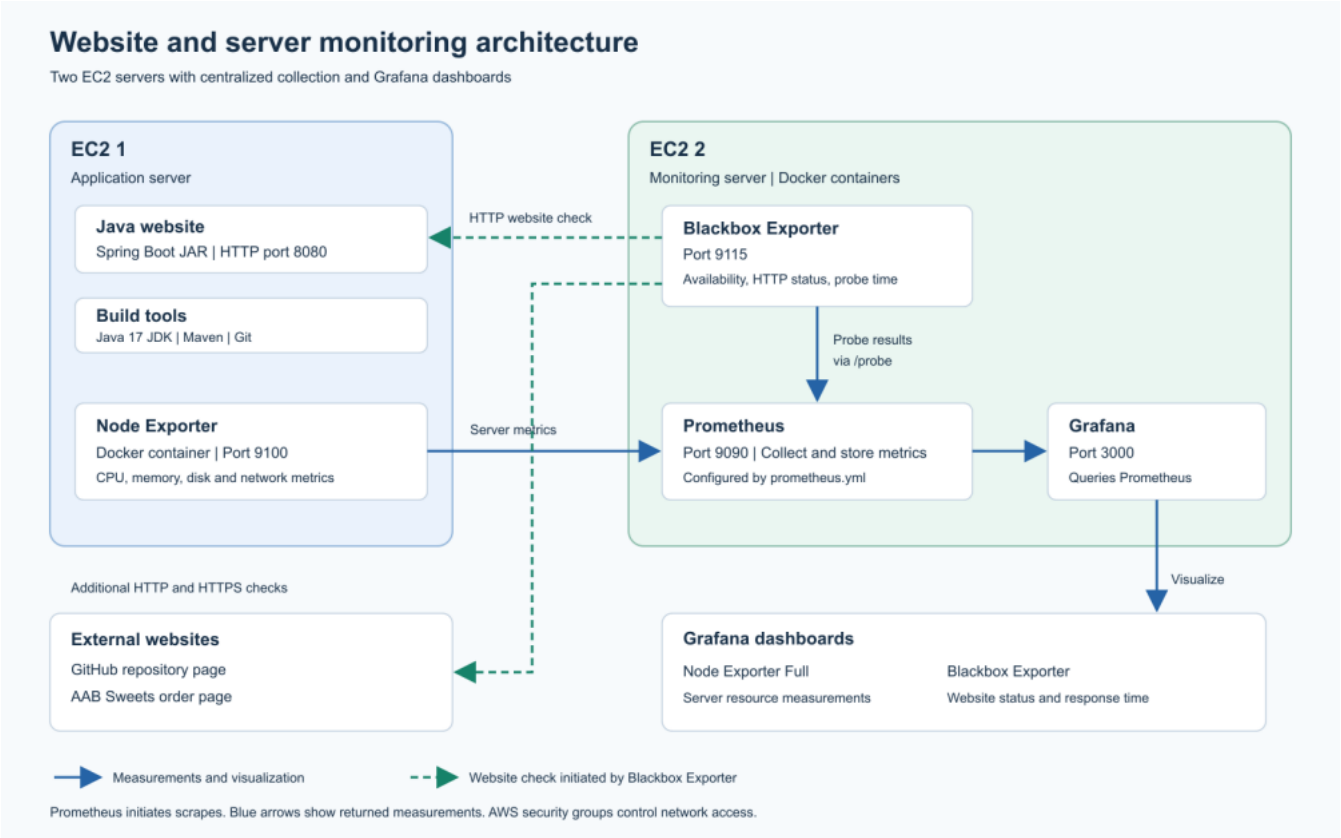


Figure 1 Architecture recreated for the completed two server deployment

Component	Purpose
Java website	Your website, opened on port 8080.
Node Exporter	Provides server information on port 9100.
Blackbox Exporter	Checks websites and provides results on port 9115.
Prometheus	Collects and saves results every 15 seconds. Opens on port 9090.
Grafana	Shows results in dashboards on port 3000.

Understand the flow

Node Exporter provides server information. Blackbox Exporter checks whether each website responds. Prometheus collects and stores their results. Grafana reads the results and shows charts.

EC2 1 runs the Java website and Node Exporter. EC2 2 runs Blackbox Exporter, Prometheus, and Grafana. Both dashboards are opened inside Grafana.

Step 1 Prepare the servers and network

Work in: AWS console. Use two Ubuntu EC2 servers. Name them EC2 1 and EC2 2 so they match the architecture. The servers must be able to connect to each other.

Server	Install or run
EC2 1	Java 17 JDK, Maven, Git, website JAR, Docker, Node Exporter
EC2 2	Docker, Blackbox Exporter, Prometheus, Grafana

Write down each server's public IP and private IP from the EC2 console. Open a separate SSH terminal for each server. SSH is the connection you use to run commands on a remote server.

Choose the addresses

Use the public IP to open a server page from your browser. This project also used public IPs between servers. If you use private IPs between the servers, use those same addresses in the configuration. A public IP may change after the server is stopped and started.

Allow the required connections

Destination	TCP port	Allowed source
Both servers	22	Your IP for connecting to the servers
EC2 1	8080	Your IP for the browser; EC2 2 for probes
EC2 1	9100	EC2 2; your IP temporarily for browser testing
EC2 2	9115	EC2 2 and your IP for browser access
EC2 2	9090	EC2 2 and your IP for browser access
EC2 2	3000	Your IP for browser access

A security group is the AWS setting that controls access to a server. Add the rules above in the EC2 console. Use your own IP for browser access. Allow the other server where the table requires it.

Step 2 Verify Java Maven and Git

Work on: EC2 1. Install Java 17, Maven, and Git. Java runs the website. The Java compiler, javac, builds Java source files. Maven runs the build. Git downloads the project.

```
sudo apt update
sudo apt install -y openjdk-17-jdk maven git
java -version
javac -version
mvn -version
git --version
```

Expected result

You should see Java 17 and javac 17. Maven should also show Java 17. The screenshot below shows the versions used in this project. Newer update numbers are normal.

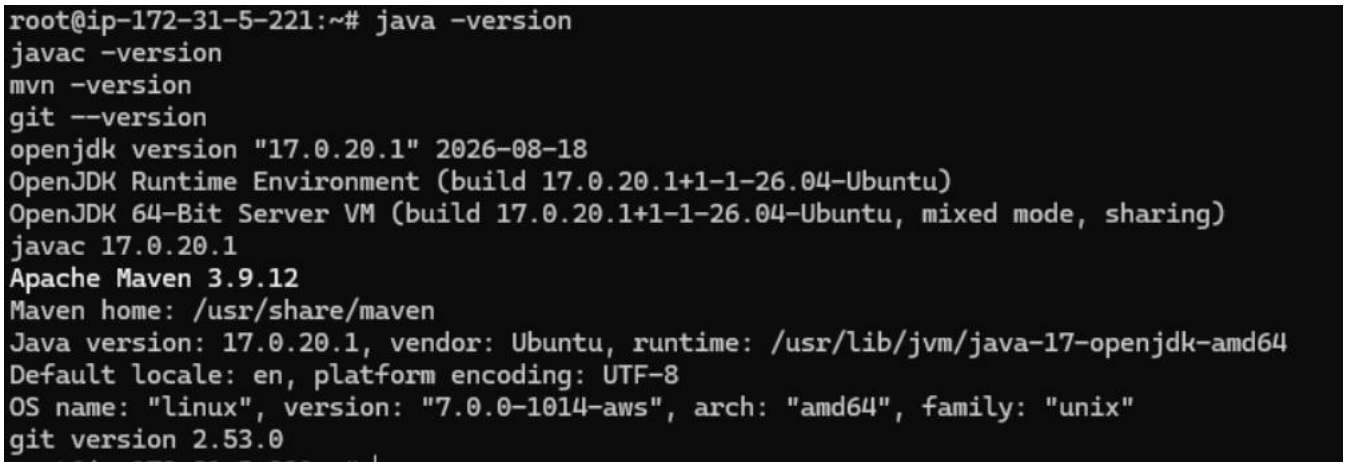


Figure 2 Verified development tools on EC2 1

If javac is missing

If javac is missing, install openjdk-17-jdk again. If Maven shows a Java version other than 17, select Java 17 before continuing.

Step 3 Clone and build the application

Work on: EC2 1. Download the project from GitHub, open its folder, and build it. If you already downloaded it, skip the git clone command.

```
git clone https://github.com/RichardL05/Prometheus_Grafana_Project.git
cd ~/Prometheus_Grafana_Project
mvn clean verify
```

If your project is in a different folder, open that folder first. Maven may download files during the first build. Wait until it finishes.

Expected result

Look for BUILD SUCCESS at the end. If you see BUILD FAILURE, fix the error before moving on.

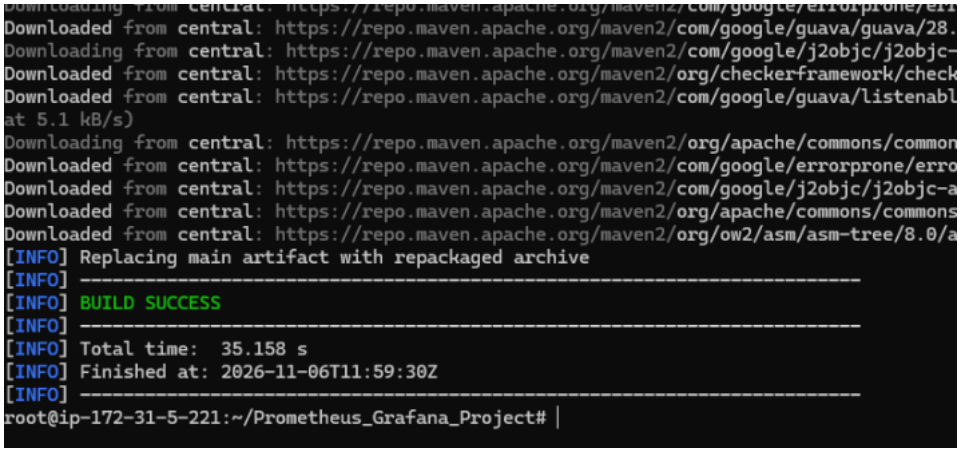


Figure 3 Successful Maven build

```
ls -lh target/*.jar
```

Use database_service_project-0.0.5-SNAPSHOT.jar to run the website. Do not use the file ending in .jar.original.

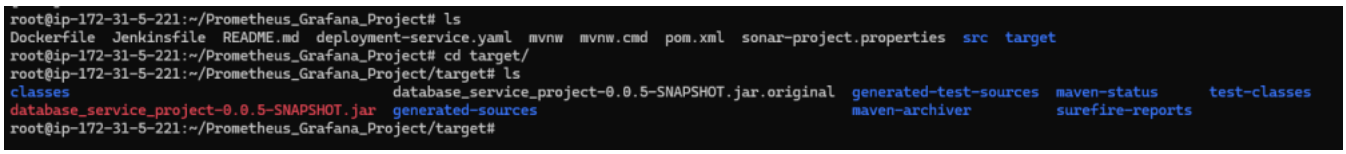


Figure 4 Generated executable JAR inside target

Step 4 Start and open the website

Work on: EC2 1. Open the target folder and run the website:

```
cd ~/Prometheus_Grafana_Project/target
java -jar database_service_project-0.0.5-SNAPSHOT.jar
```

Look for Tomcat started on port 8080 and Started DatabaseServiceProjectApplication. Keep this terminal open. Use another terminal for the next commands on EC2 1.

Open this address in your browser. Replace the label with EC2 1's public IP:

```
http://<EC2-1 App server IP>:8080/
```

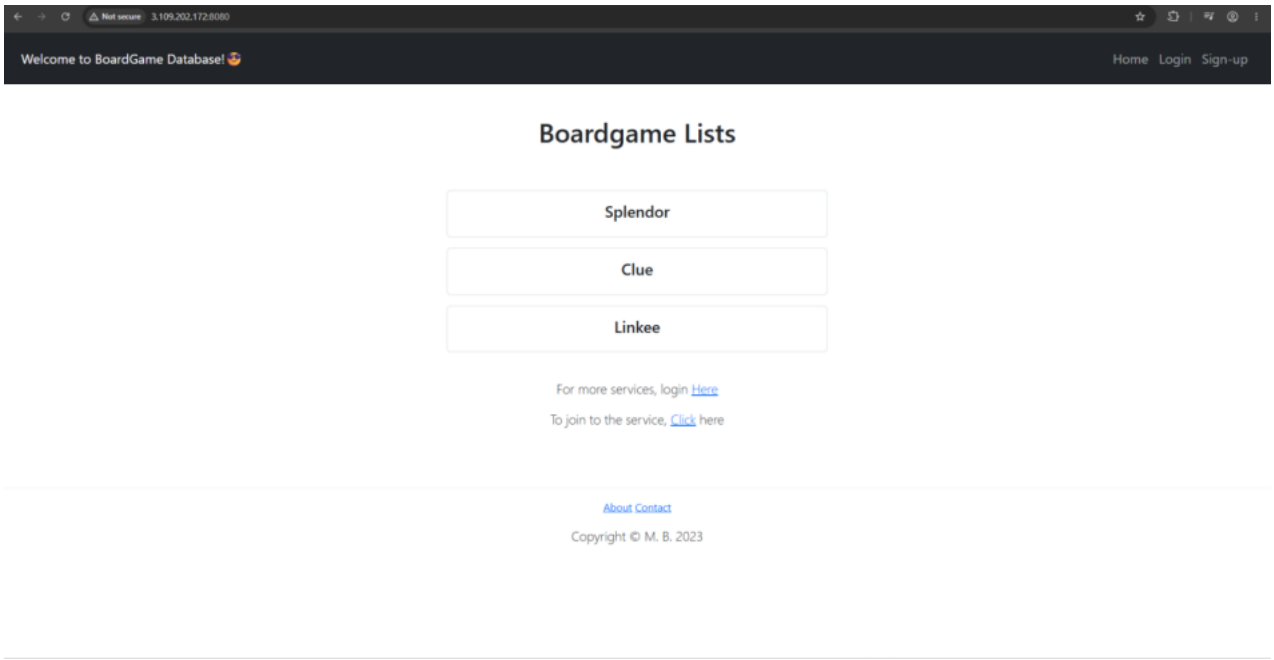


Figure 5 BoardGame website served successfully on port 8080

Keep the website running

Keep the Java terminal open while using the website. Pressing Ctrl+C stops it. If the website stops responding, check this terminal first.

Not secure appears because this website uses HTTP. HTTPS was not set up in this project.

Step 5 Install and verify Docker

Work on: both EC2 servers. Docker runs each monitoring tool in its own container. A container is a separate running copy of a tool. Install Docker on each server:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager
```

Look for active (running). The enable command also makes Docker start when the server starts.

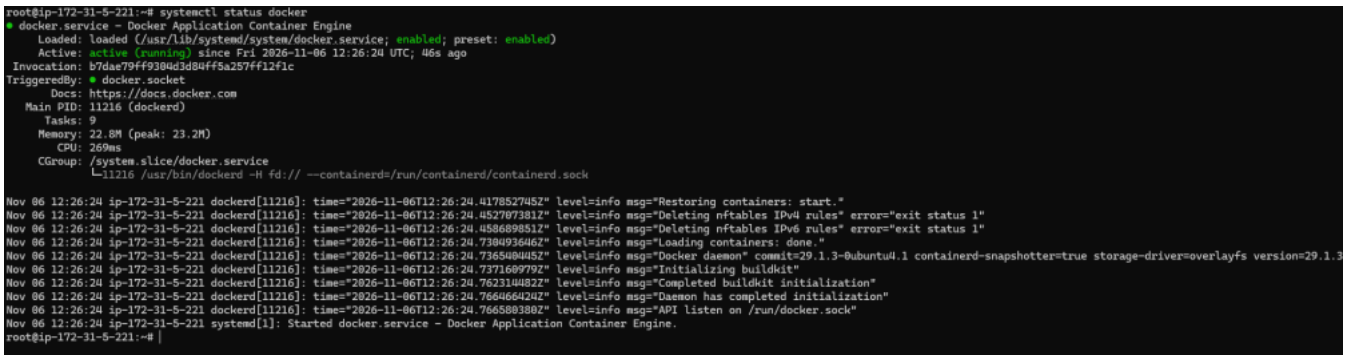


Figure 6 Docker service running and enabled on EC2 1

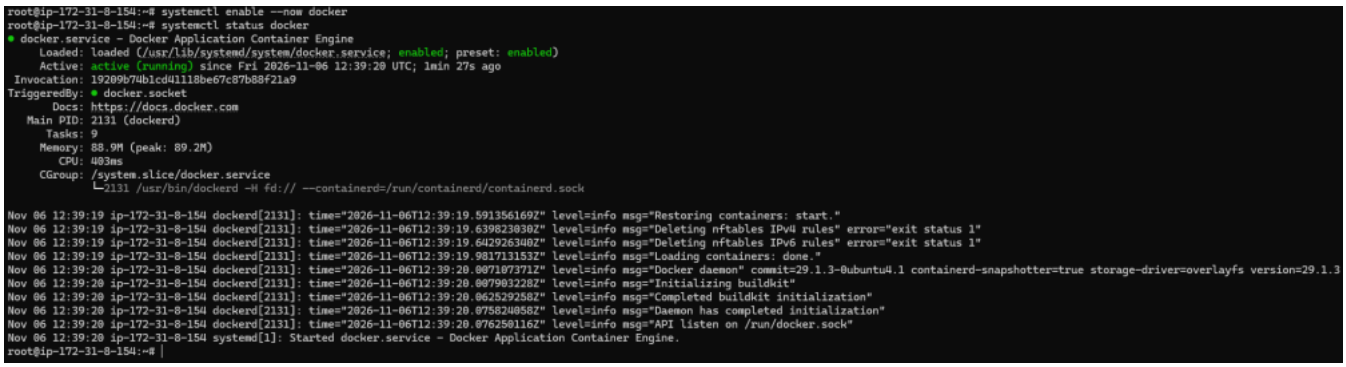


Figure 7 Docker service running and enabled on EC2 2

If a command reports Permission denied, add sudo before it. sudo runs the command with administrator access.

Step 6 Run Node Exporter

Work on: EC2 1. Start Node Exporter so Prometheus can collect server information:

```
sudo docker run -d --name=node-exporter -p 9100:9100 --restart=always --pid="host" prom/node-exporter
docker ps -a
```

Option	Meaning
-d	Run in the background.
--name=node-exporter	Name it node-exporter.
-p 9100:9100	Allow access through server port 9100.
--restart=always	Automatically restart the container, including after Docker starts.
--pid="host"	Let the exporter see the server process list.
prom/node-exporter	Use the Node Exporter software from Docker.

```
root@ip-172-31-5-221:~# docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED              STATUS              PORTS                                                  NAMES
9f52f4007870   prom/node-exporter   "/bin/node_exporter"   About a minute ago   Up About a minute   0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp   node-exporter
root@ip-172-31-5-221:~#
```

Figure 8 Node Exporter container running with port 9100 published

Open http://<EC2-1 App server IP>:9100/metrics in your browser. A page of text and numbers should appear. This is the server information Prometheus reads.

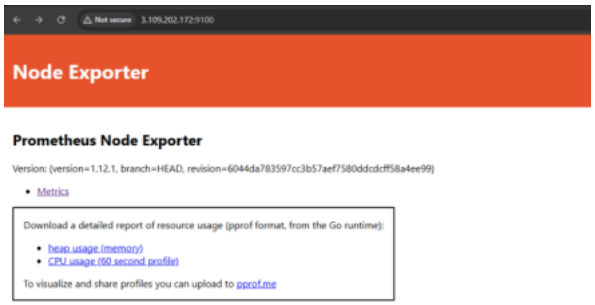


Figure 9 Accessible Node Exporter homepage

This project uses the command above. Some disk and network values can describe the Docker container instead of the full server.

Step 7 Run Blackbox Exporter

Work on: EC2 2. Start Blackbox Exporter. It checks whether your website responds and how long the check takes.

```
sudo docker run -d --name=blackbox-exporter --restart=always -p 9115:9115 prom/blackbox-exporter
docker ps -a
```

Open http://<EC2-2 Monitor server IP>:9115/ in your browser. You should see the Blackbox Exporter homepage.

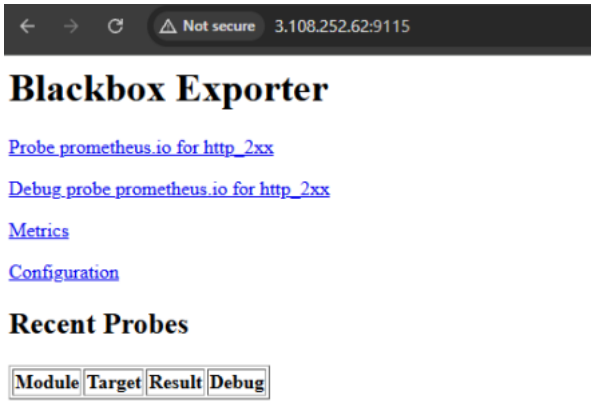


Figure 10 Blackbox Exporter homepage on EC2 2

Check your application

Replace both IP labels in the address below, then open it in your browser. Keep the Java website running on EC2 1.

```
http://<EC2-2 Monitor server IP>:9115/probe?target=http://<EC2-1 App server
IP>:8080/&module=http_2xx
```

Find probe_success. A value of 1 means the website check passed; 0 means it failed. Find probe_http_status_code. A value of 200 means the website returned a successful response.

What the measurements mean

probe_duration_seconds shows the time taken for the check, in seconds. The example link on the Blackbox homepage checks prometheus.io. Use your own website address to check your application.

Step 8 Prepare Prometheus

Work on: EC2 2. Create a file named prometheus.yml in /root. This file tells Prometheus which servers and websites to monitor.

```
sudo -i
cd /root
touch prometheus.yml
```

Open /root before running this command. $(pwd) means the folder you are currently in:

```
sudo docker run -d --name=prometheus -p 9090:9090 -v
$(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml prom/prometheus
```

Use the command above once. If the container already exists, do not run it again. The file on EC2 2 is linked to the file Prometheus reads inside its container.

Edit the settings file

Edit /root/prometheus.yml whenever you want to change what Prometheus monitors. After saving changes, restart Prometheus.

```
sudo vi /root/prometheus.yml
```

Press i to type. Paste the YAML on the next page and replace both IP labels. Use spaces for indentation. To save, press Esc, type :wq, and press Enter.

Three jobs and five targets

The file creates three groups: Prometheus itself, Node Exporter, and website checks. There are five items in total because the website group contains three URLs.

Keep the names as shown. The blackbox-exporter group checks the Java website, the GitHub project page, and the AAB Sweets page.

Step 9 Configure Prometheus targets

Work in: /root/prometheus.yml on EC2 2. Copy the text below. Replace both IP labels before saving.

```
global:
  scrape_interval: 15s
  scrape_timeout: 10s
scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets:
          - localhost:9090
  - job_name: node-exporter
    static_configs:
      - targets:
          - <EC2-1 App server IP>:9100
  - job_name: blackbox-exporter
    metrics_path: /probe
    params:
      module:
        - http_2xx
    static_configs:
      - targets:
          - http://<EC2-1 App server IP>:8080/
          - https://github.com/RichardL05/Prometheus_Grafana_Project
          - https://www.aabsweets.com/order/
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: <EC2-2 Monitor server IP>:9115
```

What the last section does

The last section sends each website address to Blackbox Exporter. It also keeps the website address as the name shown in Prometheus and Grafana. Keep these lines unchanged except for EC2 2's IP.

Verify Prometheus collection

Work on: EC2 2. Save the file, restart Prometheus, and check the file for errors:

```
sudo docker restart prometheus
sudo docker exec prometheus promtool check config /etc/prometheus/prometheus.yml
```

The check should show SUCCESS. If Prometheus stops, use the command below to read its error messages:

```
sudo docker logs --tail=50 prometheus
```

Open http://<EC2-2 Monitor server IP>:9090/targets. Wait about 30 seconds. You should see five items marked UP, as shown below.

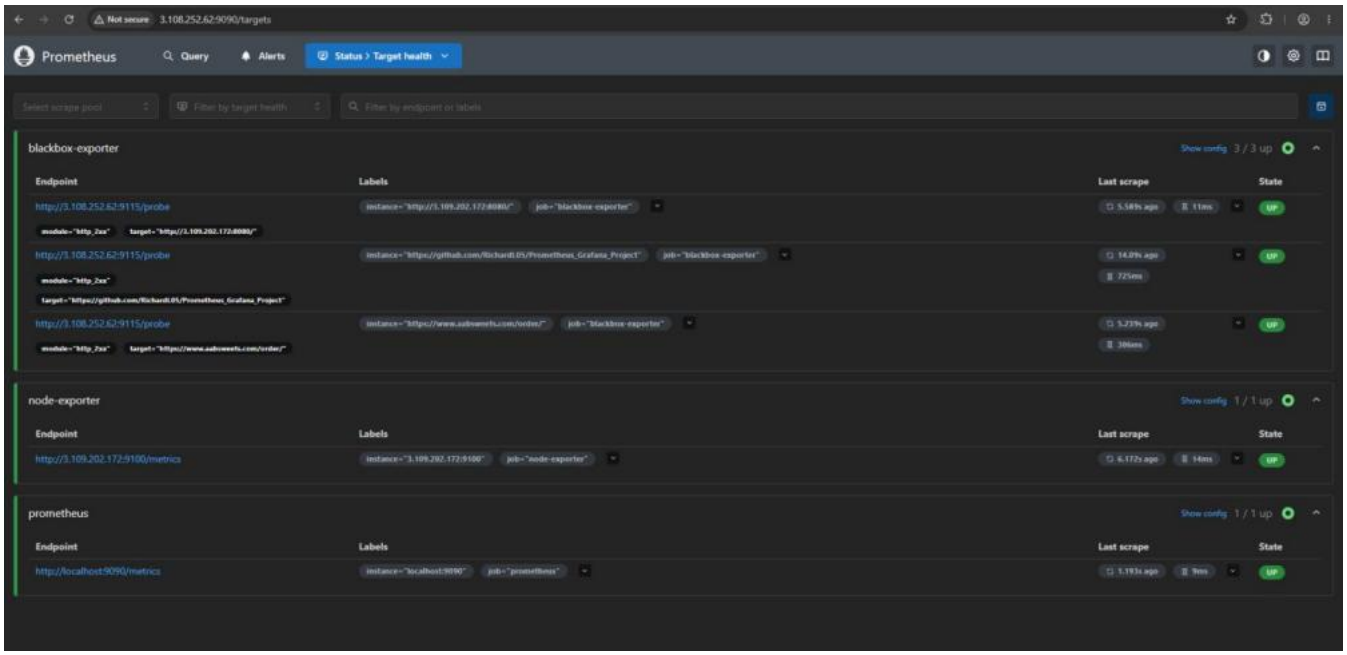


Figure 11 Five successful scrape targets across three jobs

Check website status

UP on this page means Prometheus received information. To check whether a website is working, open Query and enter:

```
probe_success{job="blackbox-exporter"}
```

For each website, 1 means the check passed and 0 means it failed.

Step 10 Start Grafana

Work on: EC2 2. Start Grafana to display your monitoring results as charts:

```
sudo docker run -d --name=grafana --restart=always -p 3000:3000 -v grafana-data:/var/lib/grafana
grafana/grafana
sudo docker ps -a
```

grafana-data saves the Grafana settings and dashboards. Keep it if you want to use the same settings later.

```
root@ip-172-31-8-154:~# docker run -d --name=grafana --restart=always -p 3000:3000 -v grafana-data:/var/lib/grafana grafana/grafana
Unable to find image 'grafana/grafana:latest' locally
latest: Pulling from grafana/grafana
f4e5155cc91f: Pull complete
1bd0660b80e5: Pull complete
4a2b3a64321a: Pull complete
9a60019a8816: Pull complete
1ff600371a0f: Pull complete
aa465df46998: Pull complete
06da073526e3: Pull complete
4f4fb700ef54: Pull complete
bcd7fa5168a7: Pull complete
7b229517ed22: Pull complete
d69c4cfa651f: Pull complete
Digest: sha256:488984b716c0862639f91a698adbe2862677a5a07bad42da5f03f3030883b205
Status: Downloaded newer image for grafana/grafana:latest
35aefc5a4d99578a5a9c6925e07a4bb0c04904386554bd1395b6de28f5fe9fca
root@ip-172-31-8-154:~# docker images

IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
grafana/grafana:latest         488984b716c0       2.01GB          503MB    U
prom/blackbox-exporter:latest  ea1c7aa3ef3f       58.3MB           19MB    U
prom/prometheus:latest         e04987641b57        389MB          116MB    U
root@ip-172-31-8-154:~# docker ps -a
CONTAINER ID   IMAGE                  COMMAND                  CREATED              STATUS              PORTS                                                  NAMES
35aefc5a4d99   grafana/grafana        "/run.sh"                About a minute ago   Up About a minute   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp            grafana
225140f13050   prom/prometheus        "/bin/prometheus --c…"   26 minutes ago       Up 4 minutes        0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp            prometheus
3c479b7484bb   prom/blackbox-exporter "/bin/blackbox_expor…"   36 minutes ago       Up 36 minutes       0.0.0.0:9115->9115/tcp, [::]:9115->9115/tcp            blackbox-exporter
root@ip-172-31-8-154:~#
```

Figure 12 Grafana Prometheus and Blackbox Exporter running on EC2 2

Sign in

Open http://<EC2-2 Monitor server IP>:3000/. For a new installation, use admin for both username and password. Set a new password when asked. If you already changed it, use your saved password.

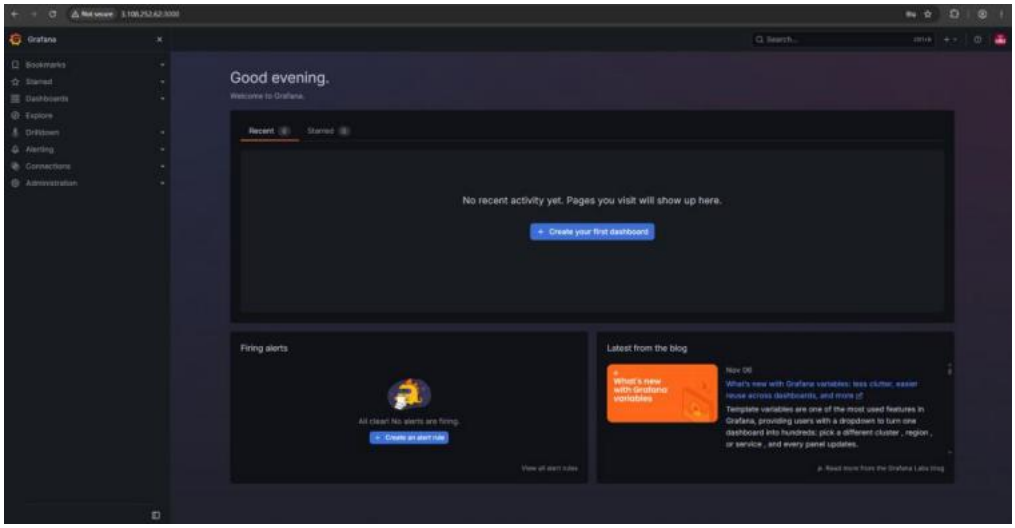


Figure 13 Grafana home after login

Step 11 Connect Grafana to Prometheus

Work in: Grafana in your browser.

1. Expand Connections in the left menu and select Data sources.

2. Select Add new data source, then choose Prometheus.

3. Set the name to prometheus and keep it as the default data source.

4. Enter http://<EC2-2 Monitor server IP>:9090/ as the Prometheus server URL.

5. Scroll to the bottom and click Save & test.

Use EC2 2's IP in this field. Do not enter localhost, because Grafana runs in a separate container from Prometheus.

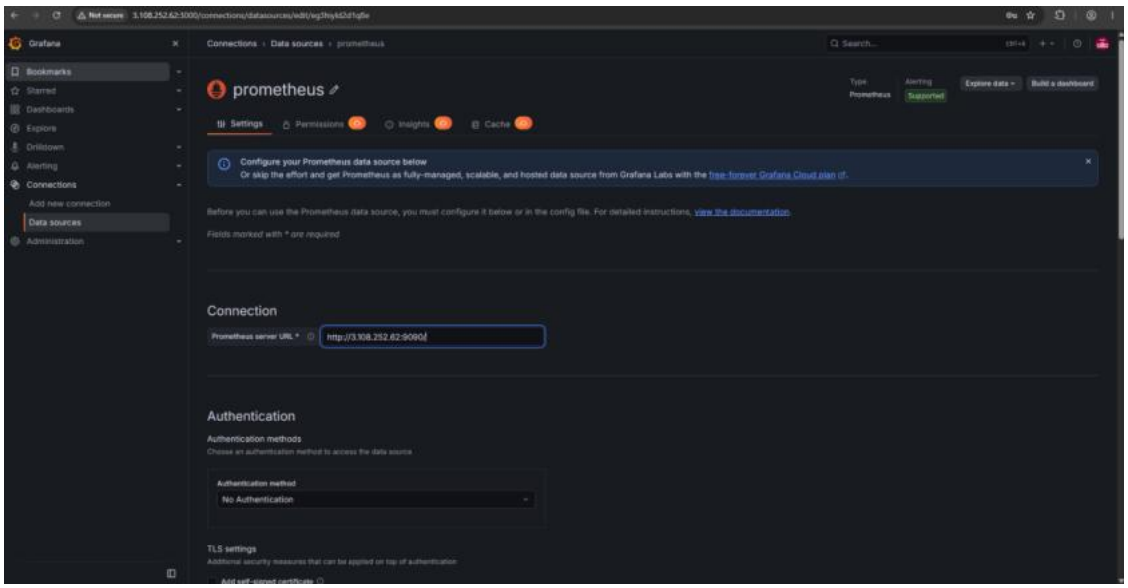


Figure 14 Prometheus data source URL configuration

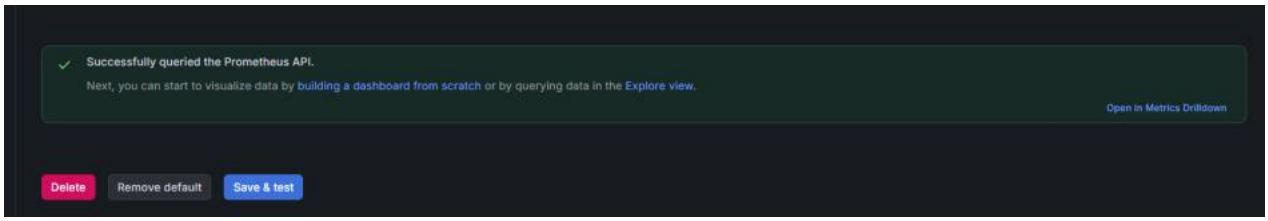


Figure 15 Successful Prometheus API test

You should see Successfully queried the Prometheus API. This means Grafana is connected to Prometheus.

Step 12 Import the server dashboard

Work in: Grafana. Both dashboards are ready-made templates from the Grafana website. Start with Node Exporter Full, ID 1860.

1. Select Dashboards, then New, then Import dashboard.

2. Enter 1860 in the Grafana.com dashboard URL or ID field and click Load.

3. Confirm the dashboard name Node Exporter Full. Select your prometheus data source if prompted.

4. Click Import.

5. Select data source prometheus, job node-exporter, and the EC2 1 instance on port 9100.

Dashboard source: https://grafana.com/grafana/dashboards/1860-node-exporter-full/

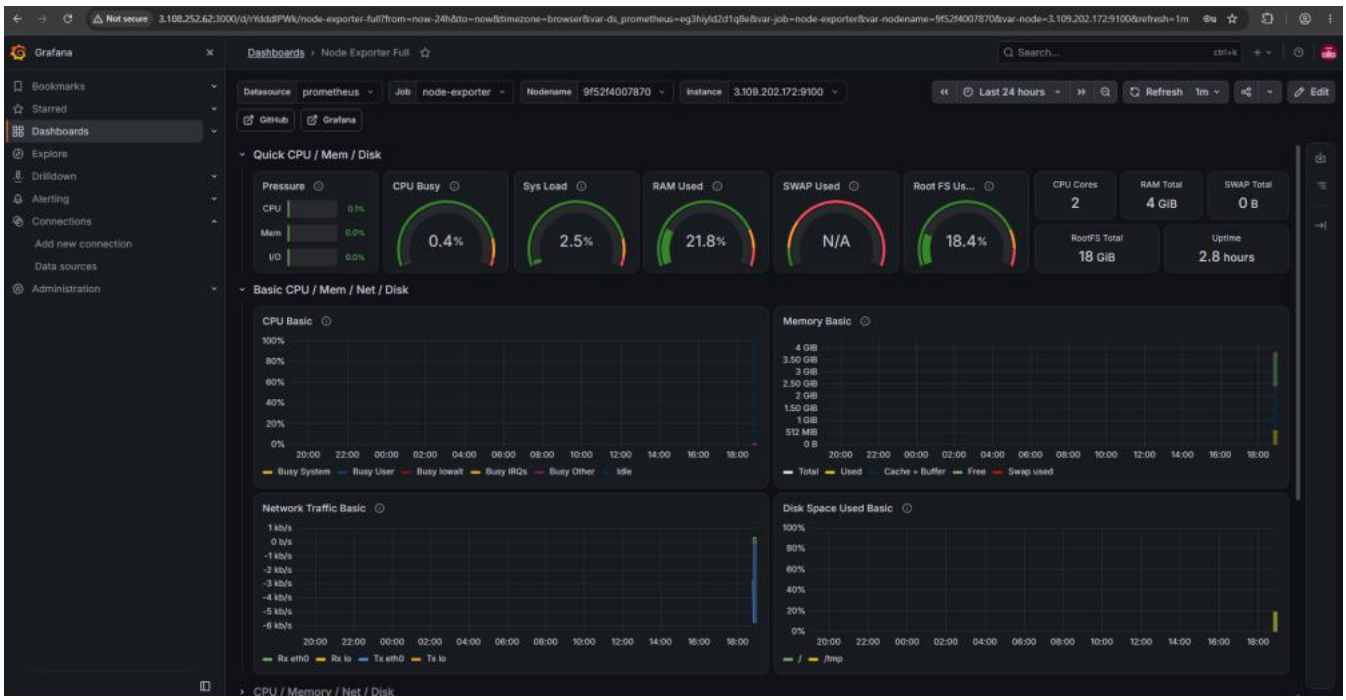


Figure 16 Imported Node Exporter Full dashboard with measurements

Read the panels

CPU Busy shows processor use. RAM Used shows memory use. Root FS Used shows disk use. Uptime shows how long the measured system has been running.

Choose Last 30 minutes to see recent results clearly. The graphs are empty for times before monitoring started. Some disk and network readings may describe the container. Swap N/A is normal here because no swap memory was configured.

Import the website dashboard

Work in: Grafana. Import Prometheus Blackbox Exporter, ID 7587, from the Grafana website. Source: https://grafana.com/grafana/dashboards/7587-prometheus-blackbox-exporter/

1. Select Dashboards, then New, then Import dashboard.

2. Enter 7587 and click Load.

3. Choose prometheus in the data source field, then click Import.

4. Select All in the target filter to show all websites.

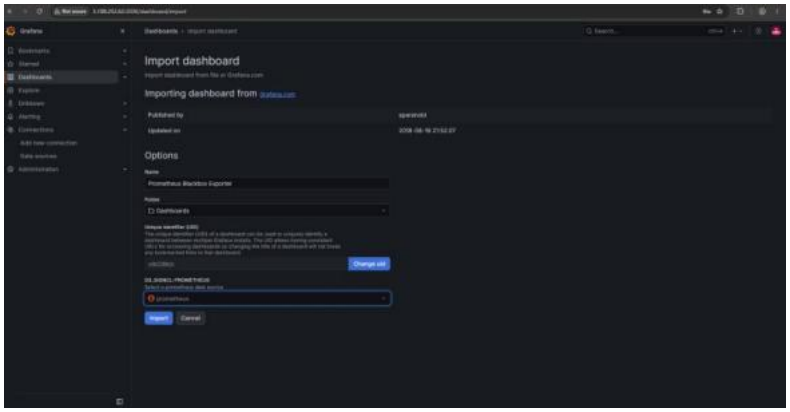


Figure 17 Map the imported website dashboard to the Prometheus data source

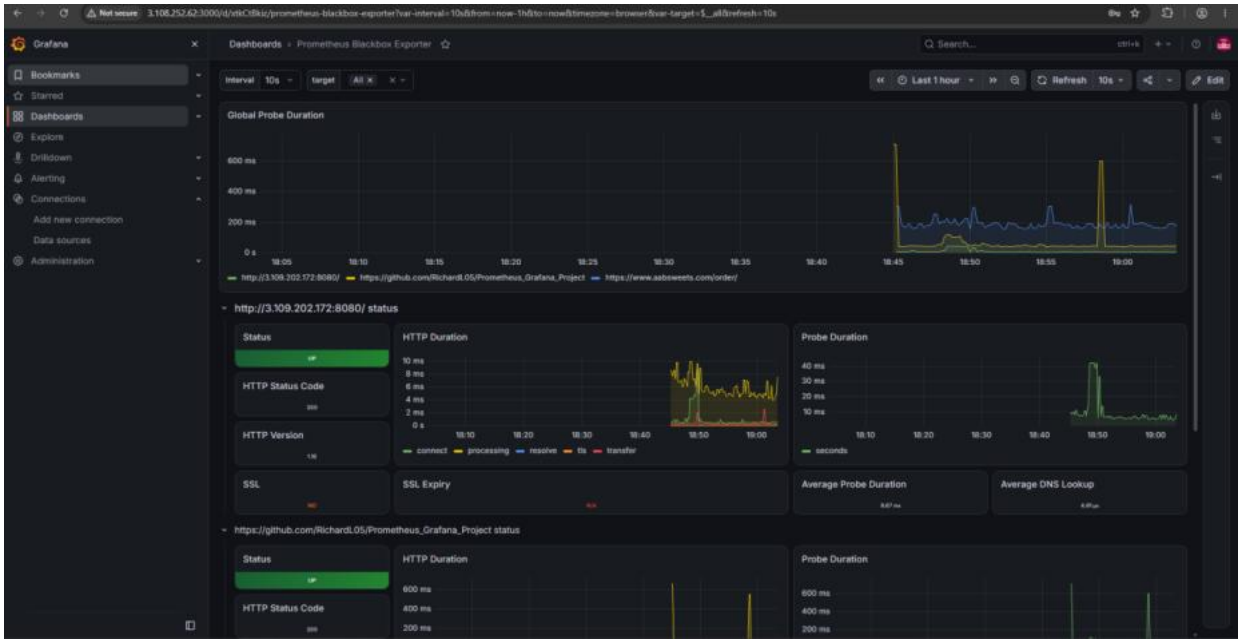


Figure 18 Website status and probe durations visualized in Grafana

UP means the website check passed. HTTP Status Code 200 means the website responded successfully. Probe Duration shows how long the check took. SSL NO and expiry N/A are normal for the Java website because it uses HTTP.

The example shows the Java website and GitHub page working. Use the target filter to view each website separately.

Issues faced and solutions

1 Docker was not installed on both servers

Problem: Running docker --version showed Command not found on EC2 1 and EC2 2.

Solution: Installed Docker and started it on each server:

```
sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager
```

Result: Docker showed active (running) on both servers. We then used it to run the monitoring tools.

2 Port 8080 was missing from the security group

Problem: The first security group settings did not include port 8080, which the Java website uses.

Solution: Opened the security group attached to EC2 1. Selected Edit inbound rules and added Custom TCP with port 8080. Saved the rule, then opened the website address in the browser:

```
http://<EC2-1 App server IP>:8080/
```

Result: The BoardGame homepage opened successfully. For a repeat setup, allow your IP for browser access and EC2 2 for monitoring.

3 Prometheus settings file was empty

Problem: Prometheus was running, but prometheus.yml was empty. The server and website addresses still needed to be added.

Solution: Added the Prometheus, Node Exporter, and Blackbox settings shown in Step 9. Saved the file and restarted Prometheus:

```
sudo docker restart prometheus
```

Then opened the Targets page:

```
http://<EC2-2 Monitor server IP>:9090/targets
```

Result: All five items showed UP: Prometheus, Node Exporter, and the three website checks.

Troubleshooting and references

Problem	Check and next action
Website does not open	Check the Java terminal, port 8080 rule, current EC2 address, and application startup logs.
Node Exporter target DOWN	Check the container on EC2 1, port 9100 access from EC2 2, and the configured address.
Website seems down but target is UP	Run probe_success in Prometheus. A value of 0 means the website check failed.
Prometheus exits after editing YAML	Run docker logs prometheus. Check the file spacing and IP addresses, then restart Prometheus.
Grafana data source test fails	Use reachable EC2 2 address and port 9090. localhost points to the Grafana container itself.
Dashboard No data	Select the correct Prometheus connection, server, and time range. Wait 30 seconds and refresh.
Container name already in use	Run docker ps -a. The container already exists. Use docker start followed by its name if it is stopped.
Disk or network panels seem wrong	Some values may describe the Docker container. Check the note in the Node Exporter step.

Useful references

Application source: https://github.com/RichardL05/Prometheus_Grafana_Project

Node Exporter configuration: https://github.com/prometheus/node_exporter

Blackbox probing: https://prometheus.io/docs/guides/multi-target-exporter/

Node dashboard: https://grafana.com/grafana/dashboards/1860-node-exporter-full/

Website dashboard: https://grafana.com/grafana/dashboards/7587-prometheus-blackbox-exporter/